LAMP: ASP Rule Induction and Optimization with LLMs

David Bunker
University of Potsdam

Headline

LAMP asks an LLM to recover ASP rules from *instance facts* and an *expected answer set*, then checks every candidate with Clingo. Across 120 synthetic rule-induction benchmarks, the strongest LAMP models nearly matched ILASP, solved every ILASP timeout case, and usually produced smaller, more stratified programs than the synthetic originals.

99.2%
Best final accuracy with LAMP
gpt-5-mini: 119 / 120

98.3%
Best local model result
gpt-oss:20b: 118 / 120

3 / 3
ILASP timeout benchmarks solved by both strong LAMP models

3–6
Average output literals for correct solutions vs. 22 or 33 originals

Problem Setting

Writing ASP that is both correct and efficient is hard. LAMP studies a focused version of that challenge: recover a rule set from example facts and the stable model it should produce.

$$AS(F \cup \Pi) = \{A^*\}$$

Here F is the given fact set, A^* is the target answer set, Φ is a bounded hypothesis space, and the goal is to find a small program $\Pi \subseteq \Phi$ that reproduces A^* exactly. Candidate programs use safe unary normal rules with bounded positive and default-negated body literals. Safe means every variable appears in a positive body literal, so grounding is finite. No new facts are allowed.

- Both LAMP and ILASP receive the same facts, target answer set, and rule bounds.
- This makes the comparison about *search strategy*: LLM-guided generation versus conflict-driven symbolic search.
- The task is intrinsically difficult: bounded-arity non-ground normal ASP reasoning is Σ_2^P-complete, and learning adds another search layer.

Toy Benchmark

Facts

```
d0(o0).
d0(o1).
d1(o1).
```

Target answer set

```
{ d0(o0), d0(o1), d1(o1), d2(o0) }
```

One minimal solution

```
d2(X) :- d0(X), not d1(X).
```

Correctness is judged by the stable model, not by reproducing the exact synthetic source rules. Many different rule sets can yield the same answer set.

Benchmark Design

Parameter	Values used
Predicates	6, 10
Terms	6, 10
Facts	5, 6, 9
Rules	11
Overall literals	2, 3 per rule body
Negative literals	0, 1, 2
Minimum derived atoms	1, 3

- 120 valid synthetic combinations were generated.
- Programs use unary predicates and normal rules only, which keeps the comparison controlled while still requiring nontrivial induction.
- Each benchmark has exactly one answer set and at least one derived atom.

Why LAMP?

- ASP is powerful for configuration, planning, and combinatorial reasoning, but writing the rules is often the bottleneck.
- LAMP uses the LLM to generate ASP rules directly from facts, a target answer set, and rule bounds.
- This differs from using LLMs mainly as a front end for natural-language translation or ILASP task construction.
- Every candidate is checked by Clingo at inference time, without LLM fine-tuning.
- Because the task is rule induction rather than text translation, success is evidence of structured reasoning rather than prompt memorization.

ILASP Baseline

ILASP solves the same task under the Learning from Answer Sets framework, but does so with conflict-driven symbolic search over an ASP metaprogram rather than iterative LLM generation.

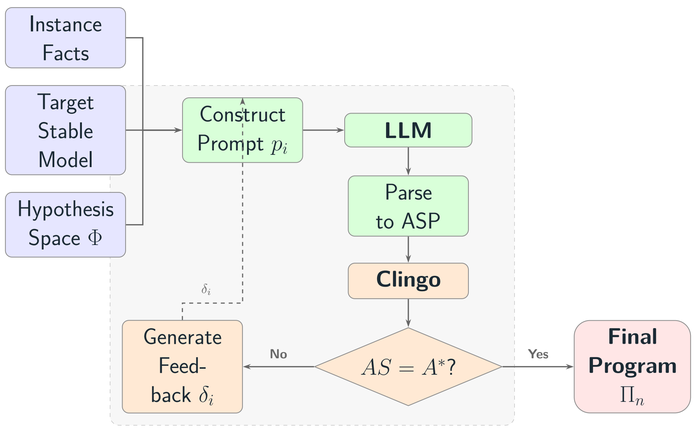
- The hypothesis space is fixed by a mode bias derived from the benchmark profile.
- The target answer set is encoded as a single context-dependent positive example.
- In this study ILASP is the main symbolic baseline, run with a 420 s timeout.

Why Hybridize?

- ILASP gives the strong symbolic baseline: usually sub-second, concise, and principled within the mode bias.
- LAMP is slower, but its token-level generation explores the space differently and solved all 3 ILASP timeout cases.
- The natural next system is not LLM versus solver; it is ILASP-style search with solver-checked LLM proposals when search stalls.

LAMP Protocol

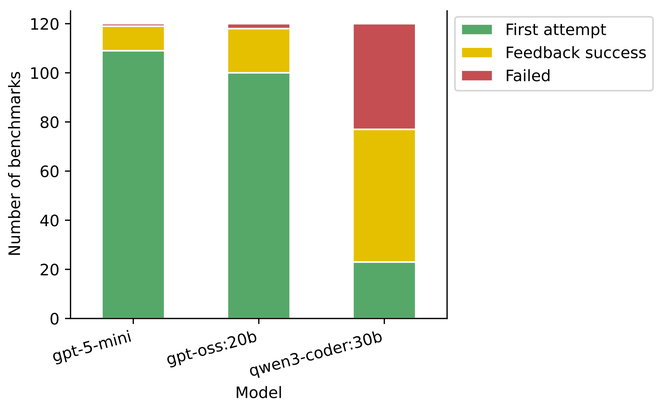
- Prompt the LLM with instance facts F, target answer set A^*, rule schema bounds, and a worked example.
- Parse the raw output into an ASP candidate program Π_i.
- Run $F \cup \Pi_i$ with Clingo.
- If the produced answer set mismatches A^*, feed the mismatch back and ask for a corrected rule set.
- Stop after success or after 3 total attempts.



The key design choice is simple: let the LLM propose rules, but let a solver decide whether those rules are actually correct.

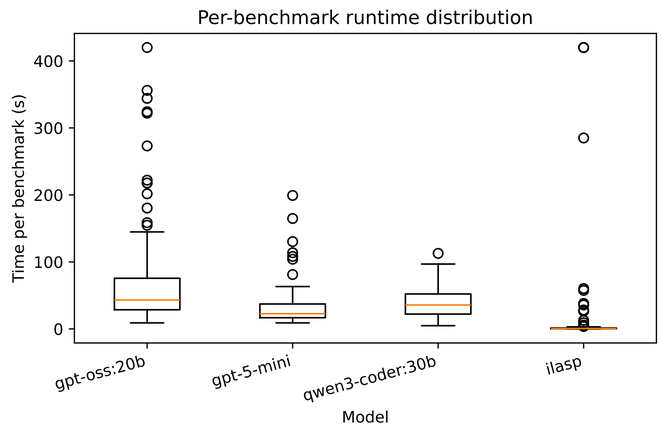
Accuracy Gains From Feedback

System	1st attempt	Final	Failures
gpt-5-mini	109 / 120	119 / 120	1
gpt-oss:20b	100 / 120	118 / 120	2
qwen3-coder:30b	23 / 120	77 / 120	43
ILASP	—	117 / 120	3



Feedback recovered 10 extra cases for *gpt-5-mini*, 18 for *gpt-oss:20b*, and 54 for *qwen3-coder:30b*. The loop matters, especially once the first draft is close but not exact.

Runtime And Search Behavior



System	Total	Mean	Median	Max
gpt-5-mini	3805.46	31.71	22.70	199.22
gpt-oss:20b	8630.91	71.92	43.22	420.00
qwen3-coder	4577.38	38.14	35.72	112.77
ILASP	1917.36	15.98	0.66	420.00

Times are seconds. LLM totals accumulate all feedback iterations; ILASP timeouts are recorded as 420 s. *gpt-5-mini* timing excludes network latency.

Program Quality

Mean output literal count $L(\Pi)$ for correct solutions

Original	gpt-5	gpt-oss	qwen	ILASP
22 literals	4.81	5.15	5.85	3.67
33 literals	3.49	3.82	3.84	2.71

Stratified solved programs

System	Stratified / Solved
Original benchmarks	49 / 120
gpt-5-mini	119 / 119
gpt-oss:20b	117 / 118
qwen3-coder:30b	77 / 77
ILASP	117 / 117

Both ILASP and strong LAMP models compressed the synthetic source programs heavily. Correct outputs usually needed only a handful of literals, and almost every solved program became stratified even when the original benchmark was not.

Timeout Cases

Benchmark	gpt-5	gpt-oss	qwen	ILASP
7	✓	✓	×	420 s
27	✓	✓	×	420 s
67	✓	✓	×	420 s

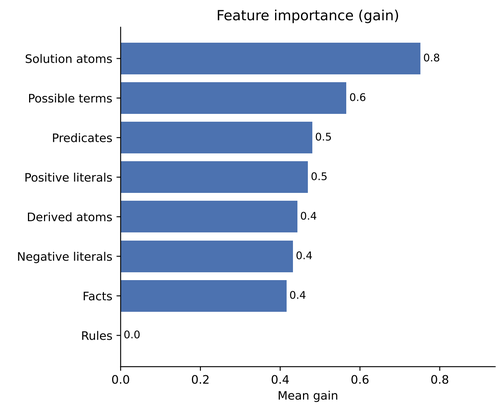
Structural profile of ILASP timeout cases

Benchmark	Grounded	Hyp. space	Neg. lits.
7	27	19906	11
27	20	16960	11
67	27	2206	11
Dataset avg	13.28	3255.22	8.80

These were timeouts, not logical contradictions. Benchmark 67 shows that grounded rules and negation can still cause search cliffs even when the hypothesis space is smaller.

Feature Analysis (RQ3)

The paper also asks which benchmark features are associated with LLM failure. The strongest models missed too few cases for a useful classifier, so the analysis focuses on *qwen3-coder:30b*.



The top gain features describe scale and vocabulary breadth: larger target answer sets, more possible terms, and more predicates make exact rule recovery harder.

XGBoost reached 61.7% cross-validation accuracy with AUC 0.619, below the 64.2% majority baseline, so these are descriptive correlates rather than reliable predictors.

Research Questions

- **RQ1:** Solver-in-the-loop prompting can produce correct ASP rules from facts and a target answer set.
- **RQ2:** LAMP nearly matches ILASP in accuracy; ILASP is faster on average, but LAMP solves the ILASP timeout cases.
- **RQ3:** Failure correlates with target size and vocabulary breadth, but the current sample is too small for reliable prediction.
- **Hybrid direction:** LAMP is not a replacement for ILASP; it is a complementary LLM-guided search strategy.

Scope And Next Steps

- The study is synthetic and restricted to unary predicates, normal rules, and exactly one answer set.
- Each model was run once per benchmark at temperature 1, so small score gaps should be interpreted cautiously.
- Next: richer ASP constructs, ASPBench or planning domains, and ILASP-style search with an agentic LLM refinement loop.

Experimental Controls

Controlled comparison

- Same input instance for both systems: facts F, target answer set A^*, and bounded rule schema.
- LAMP receives A^* in the prompt; ILASP receives it as a context-dependent positive example.
- Correctness is exact Clingo stable-model matching, so candidates are not accepted by surface similarity.

Run conditions

- 120 synthetic benchmarks over unary normal rules, with all systems limited to the same 420 s timeout.
- LAMP used 3 attempts: one initial generation and up to 2 solver-feedback refinements.
- *gpt-5-mini* used the OpenAI API; *gpt-oss:20b* and *qwen3-coder:30b* ran locally with Ollama.

Timing and data

- ILASP failures are recorded at the timeout ceiling; *gpt-5-mini* timing excludes network latency.
- Output programs are judged by their produced stable model, literal count, and stratification rather than source-rule identity.
- Source code, benchmark data, and generated ILASP tasks are available with the paper artifacts.

`https://github.com/dbunker/lamp`